Lucas Swanson
10/5/2026
ALGORITHMS

Module 1

Part 1

```
# Factors is a function that takes in an integer and returns an
array of its prime factors
# This is intended for primes that are larger than 1 based on an
input integer n. If n is less than 2, it returns an empty array.
# Concept here is to loop through all values of n where n > 2
while finding divisors of n.
# If a divisor is found, it is added to the array, and n is
divided by that divisor. This continues until n is less than 2.
def factors(n):
    if n < 2:
        return []
    factors = []
    divider = 2
    while n >= 2:
        while n % divider == 0:
            factors.append(divider)
            n //= divider
        divider += 1
    return factors
```

Part 2

Here we have a nested loop with a floor division. This has a complexity of:

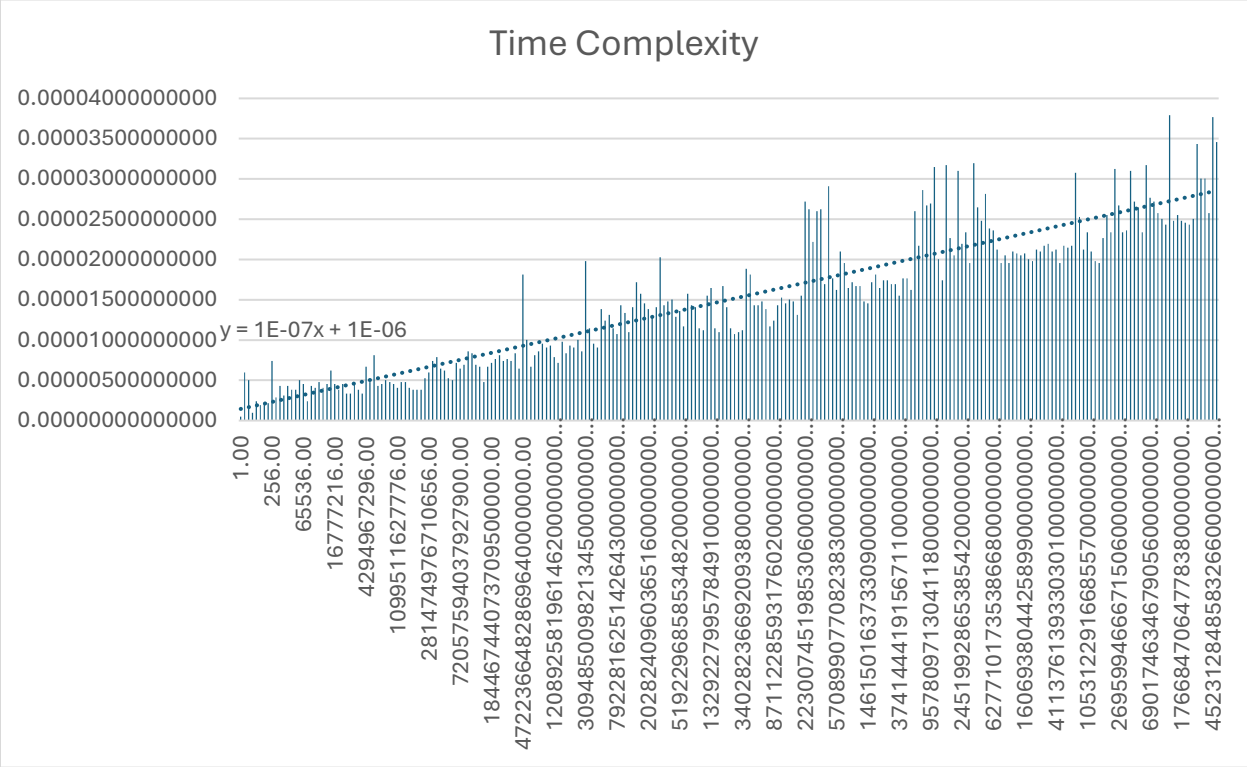
$$O(c * \mathrm{n})$$

This is assuming that the floor divider is taking constant time, and the loop only has to go through all of n once. There is no nested loops because all loops depend on the value of n that increments with each loop until they meet or n become less than 2. Lets take the following:

1. Best Case:
    a. When n → 0 (where n is still a prime number)
       As n becomes closer to 0, we are likely to take less cycles to execute.
2. Worst Case:
    a. This would be when n → ∞.
       As n gets infinitely larger, the time taken approaches infinity.

3. Average Case:
    a. Honestly the average case kind of eludes me here as the average depends on normal usage of the function. In this case there is no average case but the kinds of numbers used in the test script

Part 3



Here this shows a test case I ran from input data 1 -> $9.9e^{74}$ doubling the data every time. This perfectly shows the linear behavior of the overall time taken comparatively to the increasing size of the input data. The equation for the trendline that best fits this graph is:

$$y = 1E(-07)x + 1E(-06)$$

Using this to find the following numbers required to hit each time:

$$x = 10^7 y - 10$$

1. Y = 5 minutes (y = 300s): 3,000,000,010
2. Y = 5 hours (y = 18000s): 180,000,000,010
3. Y = 5 days (y = 432000s): 4,320,000,000,010
4. Y = 10 years (y = 315360000s): 3,153,600,000,000,010

Part 4

A) There are multiple invariants here for each loop (I have two nested while loops)

- **a.** At the beginning of every iteration of the outer loop, the array "factors" will contain only prime factors of the original input. The product of all items in the array will equal the original input.
- **b.** The second loop will continue only when a valid factor is found with a modulo with no remainder.

B) Proof

- **a. Initialization**: n is initialized when the user calls the function, and initializes an empty return array and divider parameter at 2 if the user selects a value 2 or higher.
- **b. Maintenance**: Every loop either the divider parameter increases by one, or the user input integer n is reduced with a floor division with the second loop only acting as an event handler in the case a valid divider is found for n with the modulo with no remainder being found.
- **c. Termination**: the outer loop terminates when the value of "n" falls below 2. The inner loop ends when "n % divider != 0", meaning when the current value of "divider" is not a factor of the current value of n.

Combining all invariants and items from the initialization, maintenance, and termination, we can conclude the delivered output is a sorted list of prime factors of N. This is because we know that the first loop initializes with an empty array and searches all possible factors starting with 2 and counting upwards from there (repeats are possible). Once any factor is detected with a modulo checking if the remainder is 0, which adds the current divider parameter to the list of factors, then the user input value of n is divided by the value of that found factor. The process repeats again until the input parameter of n is reduced below the value of 2 The only reason the solution is correct for an input of 1 would be the initial if statement covering any case less than 2. And any input prime number will yield the correct output because the primary secondary loop would only trigger once, when the divider parameter finally reaches the prime number and changes the parameter n to 1 and exits.